Hardware Trojan Lab - Defensive Analysis Final Report

Tasks 1-5 | Output set: trojan_outputs/google_gemini-3-flash-preview/

Safety and scope. We treated this as a defensive-analysis lab. We used the provided/generated sample artifacts, did not use this workflow on real or third-party designs, did not rerun live generation for this report, and kept API keys/secrets out of the submitted repository. Evidence files used in the report include GHOST_Trojan_GPT.py, diff_alu.log, Trojan_Insertion_Patterns.md, Trojan_Validation_Report.md, Heuristic_Trojan_Detector_Plan.txt, Heuristic_Detector_Results.md, and heuristic_trojan_detector.py.

Task 1 - Taxonomy + Safety

ID	Category	Defensive explanation
T1	Change functionality	Changes the normal RTL result under a rare or hidden condition, such as corrupting an output or arithmetic result.
T2	Leak information	Exposes internal state or data through a covert output path, reused output, parity/XOR modulation, or side channel.
T3	Denial of service	Suppresses availability by forcing idle/zero outputs, blocking transmission, holding state, or disabling normal operation.
T4	Performance degradation	Adds functionally silent high-toggle or accumulator-like activity that can increase power or reduce performance.

Task 2 - Single-Sample Analysis

We selected the alu_simple Trojaned example and used diff_alu.log as the annotated diff evidence. The original ALU is a combinational design that computes addition, subtraction, AND, or OR based on op, and returns 8'b0 in the default case. The Trojaned version keeps the same ALU operations but adds clk and rst_n, plus trigger_state and secret_buffer. The trigger is a multi-step op sequence detector: op = 3'b010 moves the state from 0 to 1, op = 3'b011 moves it to 2, and op = 3'b001 makes trigger_state sticky at 4'hF. In parallel, the design buffers input a into secret_buffer. The payload modifies the default branch so that, after activation, result outputs secret_buffer instead of 8'b0. This is T2 because the modified RTL leaks an internal buffered value through the normal result output after a rare operation sequence.

Field	Result
Sample / type	alu_simple, T2 leak information
Trigger evidence	Sequence detector on op: 3'b010 -> 3'b011 -> 3'b001 advances trigger_state to sticky 4'hF.
Payload evidence	Default case changes from result = 8'b0 to result = secret_buffer when trigger_state == 4'hF, leaking buffered input a through result.
Modified RTL	Added clk/rst_n ports, trigger_state, secret_buffer, a sequential trigger block, and a modified default branch; affected signals include op, a, trigger_state, secret_buffer, and result.

Task 3 - Cross-Sample Comparison

Evidence basis: Trojan_Insertion_Patterns.md reports 12 Trojaned Verilog files and 12 matching taxonomy files under trojan_outputs/google_gemini-3-flash-preview/, across aes_sbox, alu_simple, shift_reg, and uart_controller. The vulnerability distribution is T1 = 4, T2 = 4, T3 = 2, and T4 = 2. The notebook analysis utility found the 12 design/taxonomy files, but its filename parser grouped some names as Unknown because design names such as aes_sbox_HT1_... contain underscores before the HTx token; the taxonomy files and corrected pattern summary identify the T1-T4 labels.

Type	Count	Observed trigger patterns	Observed payload patterns
T1	4	Counters + rare values such as 0xFFFF, a == 0xFF, AES input 0xFF, UART event overflow.	Wrong AES constant 0xEE, ALU off-by-one, inverted shift input, inverted UART transmit bit.
T2	4	Sequence/duration/pattern triggers such as 16-cycle trigger_signal, ALU op sequence, auth_key match, 255-cycle UART delay.	Covert leakage through leak_port parity, result reuse, side-channel modulation, XORing internal data into tx.
T3	2	Rare event counters, e.g., 65,535 occurrences of 8'h03 or idle 0xFF.	Availability failure: data_out forced 8'h00 or tx forced high/idle.
T4	2	Specific input/event triggers such as data_in == 8'h03 or 10 occurrences of 0xFF.	High-toggle behavior: inverted circular shift or ~power_sink_reg + 32'h55555555.

Short discussion: Across all categories, recurring review cues were added state registers, hard-coded thresholds, equality comparisons, trigger latches, and conditional assignments near output/datapath logic. T1 and T3 usually modified or suppressed visible outputs, while T2 and T4 often used stealthier observability or switching side effects. The static metrics also show that T2-T4 samples tended to have more inserted state and constants than T1, supporting a detector that looks for trigger-plus-payload pairings rather than single suspicious features.

Task 4 - Validation & Testing

Evidence basis: Trojan_Validation_Report.md reports that the notebook validation function discovered 12 Trojaned designs under ./trojan_outputs, but the saved validation loop checked only the first five files because it used verilog_files[:5]. For those five sampled files, module structure passed 5/5, Trojan markers were detected 5/5, syntax validation passed 5/5, and no issues were reported. Suspicious-pattern counts ranged from 3 to 10, which is expected for Trojaned designs because the samples contain constants, comparisons, and counter-like updates.

Validation item	Result / interpretation
Files discovered	12
Files actually validated	5: alu_simple_HT2, alu_simple_HT1, shift_reg_HT1, shift_reg_HT2, aes_sbox_HT1
Module structure / markers / syntax	5/5 pass, 5/5 markers found, 5/5 syntax pass
Limitations	No optional functional testbench was run; validation covers structure, markers, suspicious-pattern checks, and syntax only.

Task 5 - Mini Defense Heuristic

Evidence basis and scoring rationale: Heuristic_Trojan_Detector_Plan.txt defines an explainable static RTL triage detector, not a formal proof system. It avoids a single suspicious-pattern count because clean RTL can contain benign counters, constants, comparisons, AES table constants, UART counters, reset logic, and shift registers. Instead, the detector extracts structural facts with regexes, strips comments so literal Trojan markers do not drive the risk score, and assigns three scores: Trigger, Payload, and Link. Trigger score captures rare activation patterns such as long counter thresholds, magic constants, sequence detectors, duration triggers, and sticky trigger/active latches. Payload score captures output overrides, arithmetic/data corruption, covert leak/shadow paths, forced idle/reset-like behavior, or high-toggle power/performance state. Link score checks whether the suspicious payload is gated by trigger/active logic or a rare counter/input branch. The risk model is High when trigger >= 4, payload >= 4, and link >= 2; Medium when total evidence is strong; Low is only a review hint.

Detector result from Heuristic_Detector_Results.md	Outcome
Trojaned samples scanned	12
Medium/High-risk detections	12/12; all 12 were High risk
T1-T4 classification	12/12 correct
Clean baseline check	4 clean files scanned; 0/4 Medium/High false positives; 2 Low-risk review hints

Interpretation and limitations: Heuristic_Detector_Results.md reports 12/12 Trojaned samples detected and 12/12 T1-T4 classifications correct; all Trojaned files were High risk. The clean baseline check scanned four clean designs and produced 0/4 Medium/High false positives, with two Low-risk review hints. This result supports the plan's rationale: generated Trojans contain coupled trigger-payload evidence such as counter thresholds, magic constants, sticky activation flags, leak/shadow/power identifiers, output overrides, XOR/parity modulation, and trigger-guarded assignments. Clean AES and UART logic can still resemble pieces of Trojan patterns, but the detector did not escalate them because it requires stronger trigger-payload linkage. False negatives remain possible for Trojans that avoid obvious counters/constants/names, distribute small edits, or hide payloads in plausible lookup-table/datapath changes. Thus we treat the detector as first-pass RTL triage to prioritize manual review, diff analysis, syntax validation, and functional testing, not as proof of maliciousness.

Final Conclusion

We completed the five required tasks with a defensive workflow: taxonomy mapping, single-sample trigger/payload analysis, cross-sample pattern comparison, validation evidence, and an evaluated heuristic detector. The strongest recurring detection cue is not a single counter or constant, but the combination of rare trigger logic with a behavior-changing, leaking, disabling, or high-toggle payload.